

Proyecto AURA

Plan de Capacitación



Integrantes:

Bruno Albín
Ignacio Villarreal
Santiago Aurrecochea
Emiliano Fau

Tutor:

Bernardo Rychtenberg

1 de julio de 2026

Índice

1	Introducción	1
2	Arquitectura de Red y Despliegue en Servidores Separados	1
2.1	Componentes de Red e Implicaciones de Seguridad	1
2.2	Funcionamiento de la Red de Contenedores (aura-network)	2
3	Entender el Sistema (Arquitectura de Software)	2
3.1	Frontend	2
3.2	API Gateway (Nginx en Backend)	3
3.3	Servicios del Backend	3
4	Operar el Sistema (Operación y Despliegue Offline)	4
4.1	Estrategia de Despliegue 100 % Offline	4
4.2	Inicialización del Stack en Servidor de Backend	6
4.3	Configuración de Certificados SSL	6
4.4	Observabilidad Offline	7
5	Modificar el Sistema (Flujos de Desarrollo)	7
5.1	Modificar Esquemas de Bases de Datos y Aplicar Migraciones	7
5.2	Agregar un Nuevo Microservicio al Stack	7
6	Escalar el Sistema (Escalabilidad)	8
6.1	Escalado Horizontal de Microservicios (Servicios Stateless)	8
6.2	Escalado Vertical de Inferencia (Ollama en Servidor GPU Único)	8
6.3	Escalado de Persistencia y Colas	9
7	Configuración de Variables de Entorno (.env)	9
7.1	Variables Críticas por Categoría	9
7.2	Procedimiento para Modificar y Aplicar Variables de Entorno	14
8	Despliegue y Orquestación con Docker Swarm (Producción)	15
8.1	Preparación del Clúster Swarm	15
8.2	Políticas de Ubicación (Placement Constraints) y Persistencia	16
8.3	Comandos Operativos de Swarm	16
8.4	Actualizaciones Progresivas con cero inactividad	16

1. Introducción

Este documento constituye el **Plan de Capacitación Técnica y Operativa** del sistema. Está dirigido al personal técnico responsable de entender, operar, modificar y mantener la plataforma.

Su objetivo es que dicho equipo adquiera el conocimiento necesario sobre la arquitectura de red y software, el despliegue 100 % offline, los flujos de desarrollo, la configuración mediante variables de entorno y la orquestación en producción con Docker Swarm.

Este plan cubre los aspectos técnicos y operativos del sistema. El uso funcional de la aplicación se describe en los manuales correspondientes (Manual de Usuario y Manual de Administración).

2. Arquitectura de Red y Despliegue en Servidores Separados

Para maximizar la seguridad física y lógica, la arquitectura de producción de AURA separa el Frontend y el Backend en servidores independientes, comunicados exclusivamente por canales cifrados internos dentro de la red corporativa.

2.1. Componentes de Red e Implicaciones de Seguridad

1. **Acceso Seguro (VPN):** Toda entrada al sistema desde el exterior de la infraestructura física del cliente debe realizarse por una conexión VPN controlada. Los DNS internos resuelven los dominios del Frontend y Backend.
2. **Servidor Frontend (Host A):**
 - **Tecnología:** Servidor físico/virtual independiente con un proceso Nginx ligero.
 - **Función:** Sirve los recursos de la aplicación Single Page Application (SPA) Angular compilada en producción.
 - **Seguridad:** Escucha únicamente por el puerto 443 (HTTPS) utilizando un certificado SSL de confianza emitido por la entidad certificadora (CA) interna de la organización.
3. **Servidor Backend (Host B):**
 - **Tecnología:** Servidor independiente de alto rendimiento (preferiblemente con GPU de arquitectura NVIDIA para inferencia).
 - **Función:** Ejecuta el stack de microservicios empaquetados en contenedores Docker mediante Docker Compose.
 - **API Gateway (Nginx):** Actúa como el único punto de contacto del backend, escuchando en el puerto 443 con terminación SSL.
 - **Configuración de CORS Estricta:** La directiva de Nginx `Access-Control-Allow-Origin` debe apuntar explícitamente y de manera única al origen del Servidor Frontend.
 - **Tránsito Seguro:** Todas las conexiones de API expuestas utilizan HTTPS, WSS (Web-Sockets seguros para chat) o Server-Sent Events (SSE) optimizados sin almacenamiento en caché (`proxy_buffering off`).

2.2. Funcionamiento de la Red de Contenedores (aura-network)

En la configuración de Docker (tanto para desarrollo local como para despliegues de producción), todos los archivos de Docker Compose comparten una única red virtual externa denominada **aura-network**.

1. **Aislamiento y Conectividad:** Esta red actúa como una LAN virtual privada dentro de Docker. Ningún contenedor de AURA se conecta a la red **bridge** por defecto; en su lugar, se asocian de manera exclusiva a **aura-network**. Esto impide la comunicación no deseada con otros contenedores del sistema operativo y ayuda a mantener el entorno aislado.
2. **Requisito de Precreación:** Dado que la red está declarada como **external: true** en los archivos de configuración de Docker Compose, este no la creará de manera automática en el comando de despliegue. Si la red no existe previamente, el inicio de los contenedores fallará.
3. **Cómo Crear la Red en el Ambiente de Desarrollo:** Si el equipo está desarrollando en una única máquina local (usando Docker clásico), debe crear la red usando el driver tipo **bridge**:

```
docker network create aura-network
```

4. **Cómo Crear la Red en el Ambiente de Producción (Swarm):** Al inicializar el despliegue en un clúster de servidores (Docker Swarm), la red debe crearse con el driver tipo **overlay** para permitir que contenedores de distintos servidores se comuniquen, y con el flag **--attachable** para facilitar labores de depuración manual:

```
docker network create -d overlay --attachable aura-network
```

3. Entender el Sistema (Arquitectura de Software)

3.1. Frontend

La interfaz de usuario está desarrollada en Angular, bajo una arquitectura modular y reactiva basada en componentes Standalone.

Estructura del Proyecto (aura-frontend/application):

- **app.routes.ts:** Define las rutas principales del sistema. Cuenta con guardias de ruta (**authenticationGuard**) para redirigir a **/login** si no existe un JWT válido.
- **core/:** Contiene interceptores HTTP (inyección de cabecera **Authorization: Bearer <token>**, manejo global de errores 401/403/500), guards de seguridad, servicios centrales (API, autenticación) y almacenamiento de estados globales con RxJS y Angular Signals.
- **features/:** Carpetas autocontenidas por módulo de negocio:
 - **chat/:** Vista principal de chat persistente e interactivo, WebSockets y transacciones de mensajes de audio.
 - **documents/:** Interfaz para la gestión y subida de archivos del usuario a colecciones.
 - **report, quiz, timeline, lessons-learned, decision-brief, checklist:** Módulos de visualización y edición en pantalla completa de artefactos generados por IA.

3.2. API Gateway (Nginx en Backend)

Centraliza la comunicación TLS, la seguridad de red y el enrutamiento hacia los microservicios en base al archivo de configuración de Nginx (`nginx.conf.template`).

Enrutamiento por Prefijo:

- `/auth/` y `/admin/` → Redirige a `aura-auth-service:8000` (Django).
- `/chat/` → Redirige a `aura-chat-service:8000` (Django Channels con Upgrade de WebSocket).
- `/llm/` → Redirige a `aura-llm-service:8000` (FastAPI).
- `/processing/` → Redirige a `aura-document-processing-service:8000` (FastAPI; `client_max_body_size` ampliado a 500M).
- `/collections/` → Redirige a `aura-document-collection-service:8000` (Django).
- `/notifications/` → Redirige a `aura-notification-service:8000` (Django; SSE optimizado con buffering desactivado).

Seguridad y Rate Limiting:

- Limita peticiones por IP usando zonas de memoria de Nginx (`limit_req_zone` de 10m).
- Cabeceras aplicadas: `Strict-Transport-Security` (HSTS), `X-Frame-Options SAMEORIGIN`, `X-Content-Type-Options nosniff`, y directivas CSP restrictivas.

3.3. Servicios del Backend

3.3.1. aura-auth-service (Django)

- **Función:** Gestor de identidad y autorización de la plataforma.
- **Integración LDAP:** Valida credenciales contra el servidor LDAP de la empresa (mediante `django-auth-ldap`). Al iniciar sesión, sincroniza los atributos del usuario (`auraClassificationLevel` y `auraCompartment`) a la base de datos PostgreSQL local.

3.3.2. aura-chat-service (Django)

- **Función:** Orquestador de salas de chat individuales y grupales.
- **Comunicación en Tiempo Real:** Utiliza Django Channels y Daphne sobre Redis (como channel layer) para mantener WebSockets asíncronos activos.
- **Procesamiento de Audio:** Cuenta con una integración nativa de `faster-whisper` (modelo `small` local) que transcribe mensajes de voz enviados por los usuarios directamente en el servidor sin recurrir a APIs externas de internet.

3.3.3. aura-document-collection-service (Django)

- **Función:** Catálogo de metadatos de documentos y colecciones creadas por los usuarios.

- **Estructura:** API REST basada en Django REST Framework con routers anidados para gestionar la pertenencia de documentos a colecciones y compartir permisos de lectura entre miembros del equipo.

3.3.4. aura-notification-service (Django)

- **Función:** Notificación en tiempo real de eventos del sistema (ej. finalización de procesamiento de un documento).
- **Arquitectura:** Utiliza Celery con Redis como broker para encolar el despacho de alertas y expone una conexión persistente por Server-Sent Events (SSE) al navegador para entrega inmediata.

3.3.5. aura-document-processing-service (FastAPI)

- **Función:** Ingesta, parseo, fragmentación y vectorización de archivos corporativos dispersos.
- **Arquitectura:** Arquitectura en capas dividida en **domain** (reglas y entidades), **application** (procesadores e ingesta), **infrastructure** (adaptadores para bases de datos y colas) y **api** (controladores HTTP).

3.3.6. aura-llm-service (FastAPI)

- **Función:** Orquesta la ejecución del LLM local de Ollama.
- **Tecnología:** Utiliza LangChain y LangGraph para diseñar flujos estructurados de conversación y RAG (recuperación inteligente basada en distancia de vectores pgvector).

4. Operar el Sistema (Operación y Despliegue Offline)

4.1. Estrategia de Despliegue 100 % Offline

Dado que los servidores no tienen conexión a internet en producción, los operadores deben aplicar la siguiente estrategia de portabilidad para levantar la infraestructura:

1. Construcción y Exportación del Frontend. El frontend también se empaqueta en una imagen de Docker para facilitar su despliegue seguro bajo Nginx. En la máquina con acceso a internet:

```
# Construir la imagen del frontend (Nginx + static compilado de Angular 20)
docker build -t aura-frontend:latest ./aura-frontend/application

# Exportar la imagen a archivo tar portable
docker save -o aura-frontend.tar aura-frontend:latest
```

Esta imagen se transfiere y carga en el **servidor frontend** usando `docker load -i aura-frontend.tar`.

2. Construcción y Exportación del Backend. Dependiendo de si el servidor cuenta con aceleración de hardware para IA, compile utilizando los archivos Compose correspondientes:

Para Variante CPU:

```
docker compose
-f docker/docker-compose/docker-compose-infrastructure.yml
-f docker/docker-compose/docker-compose-services.yml
build
```

Para Variante GPU (Recomendado para Producción):

```
docker compose
-f docker/docker-compose/docker-compose-infrastructure.yml
-f docker/docker-compose/docker-compose-services.gpu.yml
build
```

Exporte las imágenes resultantes de AURA a archivos .tar portables:

```
docker save -o aura-document-processing.tar aura-document-processing-service:latest
docker save -o aura-llm-service.tar aura-llm-service:latest
docker save -o aura-auth.tar aura-auth-service:latest
docker save -o aura-chat.tar aura-chat-service:latest
docker save -o aura-notification.tar aura-notification-service:latest
docker save -o aura-document-collection.tar aura-document-collection-service:latest
docker save -o aura-gateway.tar aura-gateway:latest
```

3. Descarga y Exportación de Terceros (Infraestructura y Observabilidad). Dado que servicios como PostgreSQL, Redis, RabbitMQ, Neo4j, Elasticsearch, Filebeat, Prometheus, Grafana y Phoenix se consumen de registros públicos de internet, sus imágenes también deben descargarse y empaquetarse previamente:

```
# Descargar imágenes de base de datos, colas y observabilidad
docker compose
-f docker/docker-compose/docker-compose-infrastructure.yml
-f docker/docker-compose/docker-compose-observability.yml
pull

# Guardar imágenes de infraestructura
docker save -o postgres.tar postgres:16-alpine
docker save -o redis.tar redis:8.2
docker save -o rabbitmq.tar rabbitmq:4.1-management
docker save -o minio.tar minio/minio:RELEASE.2025-09-07T16-13-09Z
docker save -o neo4j.tar neo4j:5-community

# Guardar imágenes del stack de observabilidad
docker save -o phoenix.tar arizephoenix/phoenix:17.4.0
docker save -o prometheus.tar prom/prometheus:v2.53.0
docker save -o elasticsearch.tar docker.elastic.co/elasticsearch/elasticsearch:8.12.0
docker save -o filebeat.tar docker.elastic.co/beats/filebeat:8.12.0
docker save -o grafana.tar grafana/grafana:10.2.3

# Guardar imágenes opcionales de LDAP de pruebas (solo para entornos de dev/test
  local offline)
# docker pull osixia/openldap:1.5.0 && docker save -o openldap.tar osixia/openldap
  :1.5.0
# docker pull osixia/phpldapadmin:0.9.0 && docker save -o phpldapadmin.tar osixia/
  phpldapadmin:0.9.0
```

Transfiera todos los archivos `.tar` al **servidor backend** y ejecute `docker load -i <nombre>.tar` para cada uno de ellos.

4. Manejo Offline de Modelos Hugging Face (Sentence-Transformers & Whisper). Los modelos de NLP y transcripción de voz se instalan directamente **durante la fase de build** en los Dockerfiles (`aura-document-processing-service/Dockerfile`). Los modelos quedan cacheados físicamente dentro de la imagen. Al levantar el contenedor en el servidor backend offline, las bibliotecas cargarán los archivos locales de caché sin buscar actualizaciones en internet.

5. Manejo Offline de Modelos de Ollama. Ollama almacena sus modelos descargados en el volumen de datos `llm_data`. Para inicializarlo offline:

- En la máquina conectada, descargue los modelos necesarios (ej. `gemma3:4b`):

```
ollama pull gemma3:4b
```

- Empaquete el directorio local de almacenamiento de Ollama (normalmente en `~/.ollama` o el directorio del volumen de docker) en un archivo comprimido.
- Transfiera y descomprima los modelos en el directorio montado como volumen `llm_data` en el Host B antes de levantar el contenedor. El healthcheck del contenedor `llm` pasará a *healthy* al detectar los archivos locales inmediatamente.

4.2. Inicialización del Stack en Servidor de Backend

Desde la raíz del repositorio en el Host B, ejecute el siguiente comando para levantar el stack completo en modo offline (utilizando CPU o la GPU local disponible):

```
# Levantar Infraestructura, Servicios y Observabilidad local
docker compose
  -f docker/docker-compose/docker-compose-infrastructure.yml
  -f docker/docker-compose/docker-compose-services.gpu.yml
  -f docker/docker-compose/docker-compose-observability.yml
up -d
```

4.3. Configuración de Certificados SSL

Nginx en el API Gateway requiere certificados válidos. Configure el montaje en el archivo de docker-compose para enlazar los certificados de la red corporativa al contenedor de Nginx:

- **Ruta de certificados:** Monte su par `certificado.crt` y `llave.key` en `/etc/ssl/certs/` del contenedor.
- **Configuración del entorno:** Modifique las variables `SSL_CERT_PATH` y `SSL_KEY_PATH` directamente en el servicio `aura-gateway` en el archivo de configuración `docker/docker-compose/docker-compose-services.yml` (y su versión `.gpu.yml`), para que apunten a los archivos internos correspondientes (por defecto apuntan a los secretos `/run/secrets/aura.crt` y `/run/secrets/aura.key`).

4.4. Observabilidad Offline

El monitoreo no requiere servicios en la nube:

- **Arize Phoenix:** Acceda a `http://localhost:6006` en el Host B (o a través del gateway si se expone) para visualizar las trazas detalladas de la ejecución del RAG, latencias de Ollama, y prompts exactos enviados.
- **Métricas y Dashboards:** Acceda a Grafana (`http://localhost:3001` - credenciales por defecto: `admin/admin`) para consultar el Dashboard de AURA, alimentado por Prometheus con métricas sobre documentos procesados con éxito, errores y tiempos por etapa.

5. Modificar el Sistema (Flujos de Desarrollo)

5.1. Modificar Esquemas de Bases de Datos y Aplicar Migraciones

Las bases de datos Postgres son inicializadas por scripts SQL iniciales, pero los cambios a esquemas existentes en producción deben aplicarse mediante migraciones incrementales e idempotentes.

1. Esquema de Procesamiento (aura-db). Los scripts de migración viven en `database/aura-db/migrations/`. Para aplicar un cambio en producción de forma manual sobre un contenedor en ejecución:

```
Get-Content database/aura-db/migrations/XXXX_nombre_migracion.sql |  
docker exec -i aura-db psql -U $env:DB_USER -d $env:DB_NAME
```

2. Esquema de Autenticación (auth-db). Los scripts viven en `database/auth-db/migrations/`. Se aplican de la misma forma para sincronizar nuevos permisos o roles en la tabla de usuarios:

```
Get-Content database/auth-db/migrations/XXXX_permissions.sql |  
docker exec -i auth-db psql -U $env:AUTH_DB_USER -d $env:AUTH_DB_NAME
```

5.2. Agregar un Nuevo Microservicio al Stack

Si necesita extender el sistema agregando un nuevo servicio interno, siga estos pasos:

1. **Crear el Contenedor:** Desarrolle el servicio (usando FastAPI o Django) e incluya un Dockerfile optimizado.
2. **Registrar en Docker Compose:** Añada la declaración del servicio en `docker-compose-services.yml`. Asegúrese de definir las variables de conexión a la infraestructura (Postgres, Redis, etc.) compartida.
3. **Configurar el Enrutamiento en Nginx Gateway:** Modifique `aura-gateway/nginx.conf.template` para añadir un nuevo bloque `location` que dirija el tráfico del prefijo asignado a su servicio:

```
location /mi-nuevo-servicio/ {  
    limit_req zone=api burst=20 nodelay;
```

```
    proxy_pass http://aura-nuevo-servicio:8000/;
}
```

4. **Validación de Autenticación:** En el código de su nuevo servicio, configure un middleware que intercepte el JWT de la cabecera `Authorization`, y valide su firma y expiración consumiendo la clave pública de `aura-auth-service` o realizando una llamada de validación HTTPS interna.

6. Escalar el Sistema (Escalabilidad)

6.1. Escalado Horizontal de Microservicios (Servicios Stateless)

Los servicios de backend de AURA son *stateless* (sin estado local) gracias a que delegan la persistencia a PostgreSQL, MinIO y Redis. Esto permite escalarlos de manera simple en producción utilizando la directiva `deploy` de Docker Compose / Swarm:

```
# Fragmento de ejemplo para escalar en el docker-compose de producción
services:
  aura-document-processing-service:
    image: aura-document-processing-service:latest
    deploy:
      replicas: 4
      resources:
        limits:
          cpus: '2'
          memory: 4G
```

Nginx balanceará automáticamente las peticiones entre los contenedores réplicas activos mediante su DNS interno de Docker.

6.2. Escalado Vertical de Inferencia (Ollama en Servidor GPU Único)

De acuerdo a las decisiones de diseño para producción, el motor de inferencia (Ollama) no se escala de forma distribuida en red, sino **de manera vertical en un único servidor con GPU de alto rendimiento**.

Para exprimir al máximo la capacidad de la GPU dedicada y evitar cuellos de botella de concurrencia:

1. **Configurar Hilos y Peticiones Paralelas:** Configure las variables de entorno del contenedor de Ollama en el compose para habilitar la concurrencia nativa de modelos en memoria:
 - `OLLAMA_NUM_PARALLEL`: Define cuántas peticiones de inferencia puede resolver el servidor de forma simultánea (el valor por defecto es 1). Ajuste a 2 o 4 dependiendo de la VRAM de la tarjeta gráfica.
 - `OLLAMA_MAX_QUEUE`: Define el tamaño de la cola de peticiones en espera cuando el procesador GPU está saturado antes de rechazar conexiones.
2. **Gestión de Memoria GPU (VRAM):** Asegúrese de limitar y ajustar los tamaños de los modelos activos. En entornos de concurrencia, usar cuantizaciones de 4 bits (`q4_K_M`) para

modelos de 8B o 12B permite mantener múltiples contextos cargados de forma concurrente en GPUs de 16GB o 24GB de VRAM sin degradar drásticamente el rendimiento del asistente.

6.3. Escalado de Persistencia y Colas

1. **RabbitMQ:** Ajuste los límites de prefetch de los consumidores en `aura-document-processing-service` para evitar que un solo worker offline intente procesar demasiados archivos pesados de manera simultánea, distribuyendo el procesamiento de forma equitativa.
2. **Celery:** Aumente el número de Celery workers y configure la concurrencia interna (`celery -A app worker --concurrency=N`) en `aura-notification-service` y `aura-chat-service` para despachar notificaciones y tareas pesadas en paralelo.

7. Configuración de Variables de Entorno (.env)

AURA utiliza clases de configuración de entorno tipadas e inicializadas de manera perezosa mediante `pydantic-settings` (en FastAPI) y `python-decouple` (en Django).

En el entorno de **desarrollo local**, cada servicio lee su configuración de un archivo `.env` en su raíz. En el entorno de **producción (Docker)**, las variables se inyectan a través de los ficheros de referencia (como `.env.docker` o `.env.docker.gpu`) declarados en el bloque `env_file` de los manifiestos de Docker Compose.

7.1. Variables Críticas por Categoría

7.1.1. Persistencia e Infraestructura

- `DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD` (Django):
 - **Ubicación:**
 - `aura-auth-service/.env.docker` (apunta a la base de datos de identidad `auth_db`).
 - `aura-chat-service/env/.env.docker` (apunta a la base de datos `aura_db`).
 - `aura-document-collection-service/.env.docker` (apunta a la base de datos `aura_db`).
 - `aura-notification-service/.env.docker` (apunta a la base de datos `aura_db`).
 - **Función:** Credenciales y dirección del contenedor de base de datos Postgres.
 - **Nota para Auth Service:** Además de la base de datos de identidad, `aura-auth-service` requiere acceso directo a la base de datos `aura_db` a través de las variables `AURA_DB_HOST / _PORT / _NAME / _USER / _PASSWORD` para realizar verificaciones de datos locales de usuarios.
- `DATABASE_MANAGER_HOST / _PORT / _USER / _PASSWORD / _NAME / _DRIVER` (FastAPI):
 - **Ubicación:** `aura-document-processing-service/env/.env.docker`
 - **Función:** Datos de conexión asíncrona hacia Postgres.
 - **Recomendación:** El driver en `DATABASE_MANAGER_DRIVER` debe ser `postgresql+asyncpg` para habilitar operaciones asíncronas optimizadas con `pgvector`.
- `REDIS_CLIENT_URL` (FastAPI) o `REDIS_URL` (Django):
 - **Ubicación:**

- aura-document-processing-service/env/.env.docker
- aura-llm-service/env/.env.docker
- aura-chat-service/env/.env.docker (REDIS_URL)
- aura-document-collection-service/.env.docker (REDIS_URL)
- aura-notification-service/.env.docker (REDIS_URL)
- **Función:** URI de conexión al broker de Redis utilizado para caché, lock de concurrencia y Django Channels (layer).
- **Ejemplo:** redis://memory_db:6379/0
- RABBITMQ_MANAGER_URL (FastAPI) o CELERY_BROKER_URL (Django):
 - **Ubicación:**
 - aura-document-processing-service/env/.env.docker
 - aura-notification-service/.env.docker (CELERY_BROKER_URL)
 - **Función:** Dirección AMQP de conexión a RabbitMQ para orquestar la ingesta asíncrona de documentos y la cola de Celery.
 - **Ejemplo:** amqp://aura_root:aura_password@queue:5672/
- MINIO_MANAGER_ENDPOINT / _ACCESS_KEY / _SECRET_KEY:
 - **Ubicación:** aura-document-processing-service/env/.env.docker
 - **Función:** Dirección y credenciales del Object Storage interno de MinIO (almacenamiento de archivos binarios).
 - **Ejemplo:** storage:9000
- NEO4J_MANAGER_URI / _USER / _PASSWORD / _DATABASE (FastAPI) o NEO4J_HTTP_URL / _HTTP_USER / _HTTP_PASSWORD (Django):
 - **Ubicación:**
 - aura-document-processing-service/env/.env.docker
 - aura-auth-service/.env.docker (NEO4J_HTTP_*)
 - **Función:** Dirección y credenciales de acceso al motor de base de datos orientada a grafos Neo4j para extraer grafos de conocimiento.
 - **Ejemplo:** neo4j://neo4j:7687

7.1.2. Directorio Activo / LDAP (Sincronización MAC)

Estas variables gobiernan la integración con el servidor de identidades corporativo.

- **Ubicación:** aura-auth-service/.env.docker
- **Variables:**
 - LDAP_SERVER_URI: Dirección IP o DNS interna del servidor LDAP.
 - LDAP_BIND_DN: DN de la cuenta administradora utilizada para realizar búsquedas de usuarios (ej. cn=admin,dc=aura,dc=local).
 - LDAP_BIND_PASSWORD: Contraseña secreta de la cuenta de Bind.
 - LDAP_USER_SEARCH_BASE: Ruta en la estructura jerárquica de LDAP donde residen las cuentas de usuario (ej. ou=users,dc=aura,dc=local).
 - LDAP_ATTR_UID / LDAP_ATTR_MAIL / LDAP_ATTR_DISPLAY_NAME: Atributos del esquema LDAP para mapear el identificador del usuario, correo y nombre visible (ej. uid, mail, displayName).

- **Mapeo de Atributos de Control de Acceso Mandatorio (MAC):**

- **LDAP_ATTR_CLASSIFICATION_LEVEL:** Define qué atributo del esquema LDAP del cliente contiene el nivel jerárquico de clasificación del usuario (ej. `auraClassificationLevel`).
- **LDAP_ATTR_COMPARTMENT:** Define qué atributo de LDAP contiene los compartimentos específicos a los que pertenece el usuario (ej. `auraCompartment`).

7.1.3. Inteligencia Artificial, Modelos y Configuración del Procesador

Configuran los motores locales, el comportamiento del procesamiento de lenguaje natural y la selección dinámica de los procesadores en el backend de ingesta.

- **TEXT_SPLITTER_ACTIVE_TYPE / EMBEDDER_ACTIVE_TYPE / RERANKER_ACTIVE_TYPE (y TEXT_CLEANER_ACTIVE_TYPE):**

- **Ubicación:** `aura-document-processing-service/env/.env.docker`
- **Función:** Definen la estrategia o el motor activo para fragmentación, embedding, reordenamiento y limpieza de texto.
- **Funcionamiento en `aura-document-processing-service`:** Estas variables son leídas por clases fábrica (`TextSplitterFactory`, `EmbedderFactory`, `RerankerFactory`, `TextCleanerFactory`). En base a su valor, cargan dinámicamente la implementación requerida:
 - **TEXT_SPLITTER_ACTIVE_TYPE:** tipo de fragmentador.
 - ◊ **docling_hybrid** (predeterminado): particionado inteligente basado en la estructura lógica del documento. Utiliza y requiere variables `TEXT_SPLITTER_DOCLING_*` (como el modelo del tokenizador y la ruta de caché en la imagen).
 - ◊ **huggingface:** fragmentación semántica utilizando tokenizadores de Hugging Face. Carga `TEXT_SPLITTER_HUGGINGFACE_MODEL`.
 - ◊ **recursive:** fragmentador recursivo básico basado en límites de caracteres (`TEXT_SPLITTER_RECURSIVE_*`).
 - **EMBEDDER_ACTIVE_TYPE:** tipo de generador de embeddings vectoriales.
 - ◊ **huggingface** (predeterminado): modelo local de Sentence-Transformers. Requiere configurar variables como `EMBEDDER_HUGGINGFACE_MODEL`, `EMBEDDER_HUGGINGFACE_DEVICE`, etc.
 - ◊ **ollama:** inferencia local contra el motor de Ollama. Requiere configurar `EMBEDDER_OLLAMA_MODEL` y `EMBEDDER_OLLAMA_URL`.
 - **RERANKER_ACTIVE_TYPE:** tipo de reordenación de fragmentos devueltos.
 - ◊ **cross_encoder** (predeterminado): modelo Cross-Encoder local. Requiere configurar variables `RERANKER_MODEL_NAME`, `RERANKER_DEVICE`, etc.
 - **TEXT_CLEANER_ACTIVE_TYPE:**
 - ◊ **simple** (predeterminado): limpiador básico de caracteres especiales.
- **Importancia:** Actúan como filtros en las clases de configuración tipadas (Pydantic). Dependiendo del valor asignado, solo se validarán las variables específicas asociadas a dicha estrategia (por ejemplo, si el embedder activo es `ollama`, se validará obligatoriamente la variable `EMBEDDER_OLLAMA_MODEL`, ignorando las variables de configuración de Hugging Face). Esto permite reconfigurar la infraestructura de procesamiento (ej. alternar entre embeddings CPU locales o inferencia GPU con Ollama) modificando una sola línea en el `.env.docker` sin reescribir código.

- **OLLAMA_LLM_FACADE_MODEL_NAME:**

- **Ubicación:** `aura-llm-service/env/.env.docker`
- **Función:** Define el modelo local de Ollama que utilizará LangGraph para razonamiento e interacción de chat.
- **OLLAMA_LLM_FACADE_BASE_URL u EMBEDDER_OLLAMA_URL:**
 - **Ubicación:**
 - `aura-llm-service/env/.env.docker` (`OLLAMA_LLM_FACADE_BASE_URL`)
 - `aura-document-processing-service/env/.env.docker` (`EMBEDDER_OLLAMA_URL`)
 - **Función:** Endpoint HTTP del motor local de Ollama.
 - **Ejemplo:** `http://llm:11434`
- **TEXT_SPLITTER_HUGGINGFACE_MODEL:**
 - **Ubicación:** `aura-document-processing-service/env/.env.docker`
 - **Función:** Define el modelo Hugging Face cargado en caché local en la imagen de procesamiento de documentos para segmentar fragmentos semánticos (si se activa `TEXT_SPLITTER_ACTIVE_TYPE=huggingface`).
- **EMBEDDER_HUGGINGFACE_MODEL:**
 - **Ubicación:** `aura-document-processing-service/env/.env.docker`
 - **Función:** Define el modelo Hugging Face utilizado para generar embeddings si se configura `EMBEDDER_ACTIVE_TYPE=huggingface`.
- **EMBEDDER_OLLAMA_MODEL:**
 - **Ubicación:** `aura-document-processing-service/env/.env.docker`
 - **Función:** Especifica el modelo Ollama que se utilizará para incrustar documentos locales si se configura `EMBEDDER_ACTIVE_TYPE=ollama`.
- **RERANKER_MODEL_NAME:**
 - **Ubicación:** `aura-document-processing-service/env/.env.docker`
 - **Función:** Nombre del modelo de Reranking local utilizado para ordenar los fragmentos recuperados en el RAG (si se activa `RERANKER_ACTIVE_TYPE=cross_encoder`).
- **WHISPER_MODEL_SIZE / WHISPER_DEVICE / WHISPER_COMPUTE_TYPE / WHISPER_PRELOAD:**
 - **Ubicación:** `aura-chat-service/env/.env.docker`
 - **Función:** Configura el tamaño del modelo y dispositivo para el motor local de transcripción de voz Whisper (ej. `small`, `cpu`, `int8`, `True`).
- **NEMO_GUARDRAILS_ENABLED:**
 - **Ubicación:** `aura-llm-service/env/.env.docker`
 - **Función:** Habilita (`True`) o deshabilita (`False`) el filtrado de seguridad local de NeMo Guardrails en el backend de LLM para mitigar prompt injections.

7.1.4. Seguridad de Red, CORS y Comunicaciones

Gobiernan el enrutamiento y las políticas de comunicación interna de AURA.

- **CORS_ALLOWED_ORIGINS** (Django) o **CORS_ORIGINS** (FastAPI):
 - **Ubicación:** en todos los archivos `.env.docker` correspondientes a cada servicio.

- **Función:** Lista de orígenes frontend permitidos para peticiones HTTP/WebSocket.
- **ALLOWED_HOSTS:**
 - **Ubicación:** `.env.docker` de los servicios Django (`aura-auth-service`, `aura-chat-service`, `aura-document-collection-service`, `aura-notification-service`).
 - **Función:** Cabecera HTTP Host permitida para proteger contra ataques de suplantación de identidad. Debe contener el dominio o nombre de servicio del contenedor del backend.
- **JWT_ACCESS_LIFETIME_MINUTES:**
 - **Ubicación:** `aura-auth-service/.env.docker`
 - **Función:** Tiempo de expiración del token JWT de acceso.
- **JWT_SIGNING_KEY / JWT_ALGORITHM:**
 - **Ubicación:** `aura-auth-service/.env.docker`
 - **Función:** Clave secreta y algoritmo utilizado para firmar el token JWT de acceso. **Es mandatorio definir una firma segura en producción.**
- **SERVICE_API_KEY** (o equivalentes como `DOC_COLLECTION_SERVICE_API_KEY`):
 - **Ubicación:**
 - `aura-auth-service/.env.docker`
 - `aura-notification-service/.env.docker`
 - `aura-document-collection-service` (configurada internamente en settings).
 - **Función:** Token secreto utilizado para llamadas inter-servicio autenticadas (S2S), evitando accesos no autorizados a servicios internos que bypassan el gateway.
- **AUTHENTICATION_SERVICE_URL** (o `AUTHENTICATION_PROVIDER_AUTHENTICATION_URL`):
 - **Ubicación:** `.env.docker` de todos los microservicios dependientes (`aura-chat-service`, `aura-document-collection-service`, `aura-notification-service`, `aura-document-processing-service`, `aura-llm-service`).
 - **Función:** URL interna del microservicio de autenticación contra el que se valida el token Bearer recibido de los clientes (ej. `http://aura-auth-service:8000/auth/validate`).

7.1.5. Variables del Frontend

El Frontend, desarrollado en Angular, gestiona su configuración mediante archivos de entorno en TypeScript (`application/src/environments/`).

- **Ubicación:**
 - `application/src/environments/environment.ts` (redirige al entorno activo).
 - `application/src/environments/environment.development.ts` (desarrollo local).
 - `application/src/environments/environment.production.ts` (producción).
- **Variables configuradas:**
 - `production` (boolean): indica si la compilación es para un entorno productivo.
 - `chatApiUrl` (string): URL del microservicio de chat (`aura-chat-service`).
 - **Desarrollo:** `'http://localhost:8003'`
 - **Producción:** `'/chat'` (usa enrutamiento relativo a través del Gateway).

- **authenticationApiUrl** (string): URL de la API de autenticación (**aura-auth-service**).
 - **Desarrollo:** 'http://localhost:8002'
 - **Producción:** '' (usa el dominio raíz del Gateway).
- **documentProcessingUrl** (string): URL de procesamiento de documentos (**aura-document-processing-service**).
 - **Desarrollo:** 'http://localhost:8000'
 - **Producción:** '/processing' (enrutamiento relativo).
- **toolsApiUrl** (string): URL de herramientas de procesamiento de documentos.
 - **Desarrollo:** 'http://localhost:8000'
 - **Producción:** '/processing' (enrutamiento relativo).
- **notificationApiUrl** (string): URL del servicio de notificaciones en tiempo real (**aura-notification-service**).
 - **Desarrollo:** 'http://localhost:8004'
 - **Producción:** '/notifications' (enrutamiento relativo).
- **llmApiUrl** (string): URL del servicio de orquestación de LLM (**aura-llm-service**).
 - **Desarrollo:** 'http://localhost:8001'
 - **Producción:** '/llm' (enrutamiento relativo).
- **adminUrl** (string): URL del panel de administración Django de identidades.
 - **Desarrollo:** 'http://localhost:8002/admin'
 - **Producción:** '/admin/' (enrutamiento relativo).
- **Nota de producción:** En el entorno de producción offline, para simplificar la resolución de nombres y evitar problemas de CORS, todas las URLs del frontend se configuran como rutas relativas (/chat, /processing, /notifications, etc.). Esto permite que el API Gateway (**aura-gateway** en Nginx) resuelva y redirija internamente el tráfico al puerto correcto en el Host B de manera transparente para el navegador del cliente.

7.2. Procedimiento para Modificar y Aplicar Variables de Entorno

7.2.1. En el Servidor de Producción (servidor de backend)

Si necesita cambiar un parámetro de configuración (por ejemplo, cambiar el modelo de LLM activo de Ollama o cambiar la IP de LDAP):

1. **Acceso al Servidor:** Ingrese al servidor de backend mediante acceso SSH seguro o terminal local.
2. **Edición del Archivo:** Abra y edite el archivo **.env.docker** correspondiente al microservicio que desea modificar (ubicados en la raíz o en carpetas **env/** de cada directorio de microservicio).
 - *Ejemplo:* Use **nano aura-llm-service/env/.env.docker**, modifique el valor de **OLLAMA_LLM_FACADE_MODEL_NAME** y guarde el archivo.
3. **Reinicio de Contenedor:** Para aplicar los cambios sin apagar toda la plataforma de AURA, reinicie únicamente el contenedor afectado recargando el archivo **.env** mediante Compose:

```
# Desde el directorio con los manifiestos de Docker Compose
docker compose
```



```
-f docker-compose-infrastructure.yml
-f docker-compose-services.yml
up -d --no-deps --build aura-llm-service
```

El flag `-no-deps` evita reiniciar los servicios de los cuales este depende (como Ollama/Redis), asegurando continuidad de servicio en los demás componentes. El flag `-build` fuerza la recreación del contenedor con las nuevas variables cargadas en el entorno.

7.2.2. En el Entorno de Desarrollo Local

1. Localice el archivo `.env` en la raíz del servicio correspondiente (ej: `aura-auth-service/.env`). Si no existe, copie el archivo del directorio `env/.env` o use un `.env.docker` como plantilla.
2. Modifique las variables necesarias.
3. Detenga el proceso local en ejecución (`Ctrl+C` en la terminal) y vuelva a iniciarlo (ej: `python -m app.main` o `python manage.py runserver`). Las clases de settings de Python volverán a parsear el archivo al levantar.

8. Despliegue y Orquestación con Docker Swarm (Producción)

En el servidor de backend (Host B) en producción, los servicios se gestionan utilizando **Docker Swarm**. Esto proporciona alta disponibilidad, auto-recuperación ante fallos de contenedores y actualizaciones sin cortes de servicio.

8.1. Preparación del Clúster Swarm

Para que Swarm pueda orquestar los microservicios de AURA, el Host B debe actuar como un clúster activo:

1. **Inicialización:** Inicialice Swarm en la interfaz de red interna del Host B:

```
docker swarm init --advertise-addr <IP_INTERNA_HOST_B>
```

2. **Gestión de Secretos en Swarm:** En Swarm, las claves privadas y certificados SSL de la CA corporativa no deben montarse como volumen plano de host por motivos de seguridad. Se administran mediante la directiva `secrets` nativa (declarada en `docker-compose.swarm.yml`):

- Cargue el certificado en el almacén cifrado de Swarm:

```
docker secret create aura_cert ./certs/certificado.crt
docker secret create aura_key ./certs/llave.key
```

- Swarm se encargará de descryptar en memoria y montar estos archivos únicamente dentro del contenedor de `aura-gateway` en las rutas `/run/secrets/aura_cert` y `/run/secrets/aura_key` de forma transparente.

8.2. Políticas de Ubicación (Placement Constraints) y Persistencia

Docker Swarm gestiona servicios que pueden reubicarse o reiniciarse de forma dinámica. Sin embargo, para evitar pérdidas de datos en un stack puramente self-hosted, AURA utiliza reglas estrictas de ubicación:

1. **Restricción para Servicios Stateful (Con Estado):** PostgreSQL (`aura_db` y `auth_db`), Redis (`memory_db`), RabbitMQ (`queue`) y MinIO (`storage`) se ejecutan bajo la siguiente regla en sus respectivos manifiestos de infraestructura:

```
deploy:
  replicas: 1
  placement:
    constraints: [node.role == manager]
```

Esto asegura que el motor de Swarm nunca desplace estos contenedores a otros nodos del clúster donde no existan físicamente sus volúmenes persistentes de almacenamiento local (`aura_db_data`, etc.).

2. **Servicios Stateless (Sin Estado):** Las APIs y los workers no tienen restricciones de almacenamiento local. Swarm escala automáticamente estos servicios utilizando múltiples réplicas (`replicas: 2` definidas en `docker-compose.swarm.yml`), balanceando el tráfico de entrada gracias a su malla de enrutamiento interna (routing mesh).

8.3. Comandos Operativos de Swarm

Para desplegar y actualizar el stack de backend completo en producción en el Host B:

- **Levantar o Actualizar todo el Stack:**

```
docker stack deploy
  -c docker-compose-infrastructure.yml
  -c docker-compose-services.yml
  -c docker-compose.swarm.yml
  aura-stack
```

- **Verificar el Estado de los Servicios:**

```
docker stack services aura-stack
```

- **Monitorear los Contenedores en Ejecución:**

```
docker stack ps aura-stack
```

- **Bajar el Stack completo:**

```
docker stack rm aura-stack
```

8.4. Actualizaciones Progresivas con cero inactividad

Swarm permite al equipo de desarrollo desplegar nuevas versiones de los microservicios sin interrumpir el funcionamiento de la plataforma:

- Al ejecutar un redesplicue de **docker stack deploy** con una nueva imagen, Swarm no baja todas las réplicas juntas. Detiene la réplica 1 del servicio (ej: **aura-auth-service**), inicia el nuevo contenedor, ejecuta y aprueba su **healthcheck** de readiness, y solo entonces repite el proceso con la réplica 2. El Gateway de Nginx y la routing mesh derivan el tráfico a los contenedores saludables en tiempo real, ofreciendo cero interrupción al usuario final.